

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №1 (Яндекс.Контест)

Выполнил:

Студент группы Р3231

Тимошкин Р.В.

Преподаватель:

Косяков М.С.

Санкт-Петербург, 2025г.

Задача №1 «А. Агроном-любитель»

Описание:

Лёша вырастил n цветов в длинной грядке. Нужно найти самый длинный участок, где нет трех одинаковых цветов подряд.

Доказательство:

Проходим по всему массиву цветов по 3 элемента, если они одинаковые обнуляем текущие границы отрезка до 2 крайних элементов, иначе увеличиваем правую на 1 и проверяем максимум. В итоге сохраняется инвариант, что в рассматриваемом отрезке только 3 последних элемента могут быть одинаковыми.

Алгоритмическая сложность:

- По времени - $O(n)$.
- По памяти: - $O(1)$.

Код:

```
1. #include <cstdint>
2. #include <stdint>
3. #include <iostream>
4. #include <utility>
5. #include <vector>
6.
7. namespace {
8. auto LongestRange(const std::vector<uint64_t>& flowers) ->
    std::pair<size_t, size_t> {
9.     if (flowers.size() < 3) {
10.         return std::make_pair(1, flowers.size());
11.     }
12.
13.     std::pair<size_t, size_t> range(0, 1);
14.     std::pair<size_t, size_t> max = range;
15.
16.     for (size_t i = 2; i < flowers.size(); ++i) {
17.         if (flowers[i] == flowers[i - 1] && flowers[i - 1] ==
flowers[i - 2]) {
18.             range = std::make_pair(i - 1, i);
19.             continue;
20.         }
21.
22.         range.second++;
23.
24.         if (range.second - range.first + 1 > max.second - max.first
+ 1) {
25.             max = range;
26.         }
27.     }
28.
29.     max.first++;
```

```

30.         max.second++;
31.
32.         return max;
33.     }
34. } // namespace
35.
36. auto main() -> int {
37.     size_t n_cnt = 0;
38.
39.     std::cin >> n_cnt;
40.
41.     std::vector<uint64_t> flowers;
42.     flowers.reserve(n_cnt);
43.
44.     uint64_t current = 0;
45.     for (size_t i = 0; i < n_cnt; ++i) {
46.         std::cin >> current;
47.         flowers.push_back(current);
48.     }
49.
50.     auto answer = LongestRange(flowers);
51.     std::cout << answer.first << " " << answer.second << "\n";
52. }

```

Задача №2 «В. Зоопарк Глеба»

Описание:

Животные и их ловушки соответствуют парным символам разного регистра, нужно определить, можно ли провести непересекающиеся дуги внутри круга и восстановить ответ.

Доказательство:

Задача на скобочки для самых маленьких, нужно чтобы любая пара животное-ловушка не содержала элемент без пары внутри. Варианта 2 - либо пара на другом конце, либо по соседству. Иначе конец алгоритма.

Алгоритмическая сложность:

- По времени - $O(n)$.
- По памяти: - $O(n)$.

Код:

```

1. #include <cstdint>
2. #include <iostream>
3. #include <list>
4. #include <optional>
5. #include <string>
6. #include <utility>
7. #include <vector>
8.
9. constexpr char AsciiCaseDiff = 32;

```

```

10.
11.     namespace {
12.         auto CatchAnimals(const std::string& animals) ->
            std::optional<std::vector<size_t>> {
13.             std::vector<size_t> answer(animals.size() / 2);
14.
15.             std::list<std::pair<size_t, char>> queue;
16.
17.             size_t count = 0;
18.             for (size_t i = 0; i < animals.size(); i++) {
19.                 if (std::isupper(animals[i]) != 0) {
20.                     queue.emplace_back(i - count, animals.at(i));
21.                 } else {
22.                     queue.emplace_back(++count, animals.at(i));
23.                 }
24.             }
25.
26.             while (!queue.empty()) {
27.                 const auto current = queue.front();
28.                 const auto next = ++queue.begin();
29.                 const auto last = queue.back();
30.
31.                 if (std::abs(current.second - last.second) == AsciiCaseDiff)
32.                 {
33.                     const auto trap = std::isupper(current.second) != 0 ?
34.                         current : last;
35.                     const auto animal = std::isupper(current.second) != 0 ?
36.                         last : current;
37.                     answer[trap.first] = animal.first;
38.                     queue.pop_front();
39.                     queue.pop_back();
40.                 } else if (std::abs(current.second - next->second) ==
41.                     AsciiCaseDiff) {
42.                     const auto trap = std::isupper(current.second) != 0 ?
43.                         current : *next;
44.                     const auto animal = std::isupper(current.second) != 0 ?
45.                         *next : current;
46.                     answer[trap.first] = animal.first;
47.                     queue.pop_front();
48.                     queue.pop_front();
49.                 } else {
50.                     break;
51.                 }
52.             }
53.
54.             if (!queue.empty()) {
55.                 return std::nullopt;
56.             }
57.         }
58.     }

```

```

53.
54.     return answer;
55. }
56. } // namespace
57.
58. auto main() -> int {
59.     std::string input;
60.     std::cin >> input;
61.
62.     // No monadic operations ??? :(
63.     if (auto answer = CatchAnimals(input)) {
64.         std::cout << "Possible\n";
65.         for (const auto trap : *answer) {
66.             std::cout << trap << " ";
67.         }
68.     } else {
69.         std::cout << "Impossible";
70.     }
71.
72.     std::cout << "\n";
73. }

```

Задача №3 «С. Конфигурационный файл»

Описание:

Нужно обработать конфигурационный файл, где строки могут задавать переменные и их значения, а также открывать/закрывать блоки фигурными скобками. При закрытии блока необходимо восстанавливать старые значения переменных.

Доказательство:

Мы храним текущее состояние всех переменных в хеш-таблице для быстрого доступа, а также поддерживаем стек изменений. Каждый раз при присвоении сохраняется предыдущее состояние переменной, чтобы при выходе из блока откатить все изменения. Это обеспечивает корректную работу со вложенными блоками и не даёт пропустить ни одно изменение.

Алгоритмическая сложность:

- По времени - $O(n)$.
- По памяти: - $O(n)$.

Код:

```

1. #include <iostream>
2. #include <list>
3. #include <string>
4. #include <unordered_map>
5.
6. namespace {
7.     auto Split(const std::string& input, char delimiter) ->
        std::pair<std::string, std::string> {
8.         const auto pos = input.find(delimiter);
9.         if (pos != std::string::npos) {

```

```

10.         const auto part1 = input.substr(0, pos);
11.         const auto part2 = input.substr(pos + 1);
12.         return std::make_pair(part1, part2);
13.     }
14.     return std::make_pair(input, "");
15. }
16.
17. struct Change {
18.     std::string name;
19.     std::string oldValue;
20.     bool existed;
21. };
22. } // namespace
23.
24. auto main() -> int {
25.     std::unordered_map<std::string, std::string> current_scope;
26.     std::list<int> block_changes;
27.     std::list<Change> changes;
28.     block_changes.push_front(0);
29.
30.     for (std::string input; std::getline(std::cin, input);) {
31.         if (input == "{") {
32.             block_changes.push_front(0);
33.         } else if (input == "}") {
34.             int num = block_changes.front();
35.             block_changes.pop_front();
36.             while ((num--) != 0) {
37.                 auto& change = changes.back();
38.                 if (!change.existed) {
39.                     current_scope.erase(change.name);
40.                 } else {
41.                     current_scope[change.name] = change.oldValue;
42.                 }
43.                 changes.pop_back();
44.             }
45.         } else {
46.             auto [name, value] = Split(input, '=');
47.             bool existed = (current_scope.find(name) !=
current_scope.end());
48.             std::string old_val = existed ? current_scope[name] : "";
49.             block_changes.front()++;
50.
51.             if (!value.empty() && (std::isdigit(value.front()) == 0) &&
value.front() != '-') {
52.                 auto scope_value = current_scope.find(value);
53.                 if (scope_value == current_scope.end()) {
54.                     value = "0";
55.                 } else {
56.                     value = scope_value->second;
57.                 }

```

```

58.         std::cout << value << "\n";
59.     }
60.
61.     changes.emplace_back(name, old_val, existed);
62.     current_scope[name] = value;
63. }
64. }
65. }

```

Задача №4 «D. Профессор Хаос»

Почему работает:

Линейное вычисление $a*b-c$, но с небольшой оптимизацией:

Если значение `current_count` достигло `d`, во все последующие дни оно тоже будет `d`, аналогично при `current_count = prev*b-c`, поэтому при совпадении с предыдущим числом можно завершать алгоритм.

Алгоритмическая сложность:

- По времени - $O(n)$.
- По памяти: - $O(1)$.

Код:

```

1.  #include <cstdint>
2.  #include <iostream>
3.  #include <limits>
4.
5.  auto main() -> int {
6.      size_t a{};
7.      size_t b{};
8.      size_t c{};
9.      size_t d{};
10.     size_t k{};
11.
12.     std::cin >> a >> b >> c >> d >> k;
13.
14.     size_t prev = std::numeric_limits<size_t>::max();
15.     size_t current_count = a;
16.
17.     while ((k--) != 0) {
18.         current_count *= b;
19.         if (current_count < c) {
20.             current_count = 0;
21.             break;
22.         }
23.         current_count -= c;
24.         current_count = std::min(current_count, d);
25.         if (current_count == prev) {

```

```
26.         break;
27.     }
28.     prev = current_count;
29. }
30.
31.     std::cout << current_count << '\n';
32. }
```